

Compiladores

Patito - entrega #5

Carlos Fernando Ramos Mena

Matrícula: A01197622

Tec de Monterrey

8 de junio de 2026

1. Introducción

A continuación se adjuntan las bases incluidas en el entregable 0 mismas que serán usadas en próximos entregables.

2. Definición de Tokens

Token	Significado	Regex
TOKEN_PROGRAM	Indica inicio de un programa	PROGRAM
TOKEN_START	Indica el inicio de un cuerpo	BEGIN
TOKEN_END	Indica el fin de un cuerpo	END
TOKEN_VARS	Indica el inicio de una variable	VAR
TOKEN_INT	Tipo entero	INT
TOKEN_FLOAT	Tipo flotante	FLOAT
TOKEN_PRINT	Indica inicio de impresión	PRINT
TOKEN_WHILE	Indica inicio de ciclo	WHILE
TOKEN_DO	Indica inicio del cuerpo de un ciclo	DO
TOKEN_IF	Indica inicio de un condicional	IF
TOKEN_ELSE	Indica inicio de condicional else	ELSE
TOKEN_VOID	Retorno nulo de una función	VOID
TOKEN_RETURN	Retorno de una función	RETURN
TOKEN_ID	Identificador	[a-zA-Z_] [a-zA-Z0-9_]*
TOKEN_LETRERO	Letrero a imprimir en PRINT	"[^"]*"
TOKEN_CTE_INT	Constante entera	[0-9]+
TOKEN_CTE_FLOAT	Constante flotante	[0-9]+.[0-9]+
TOKEN_SEMICOLON	Terminación de instrucción	;
TOKEN_COMMA	Separador de continuidad	,
TOKEN_COLON	Definición	:
TOKEN_LBRACE	Inicio de bloque	{
TOKEN_RBRACE	Fin de bloque	}
TOKEN_LBRACKET	Inicio de delimitación	[
TOKEN_RBRACKET	Fin de delimitación	]
TOKEN_LPAREN	Inicio de paréntesis	(
TOKEN_RPAREN	Fin de paréntesis	)
TOKEN_SUM	Suma	+
TOKEN_SUBS	Resta	-
TOKEN_DIV	División	/
TOKEN_TIMES	Multiplicación	*
TOKEN_EQUALS	Asignación	=
TOKEN_GT	Mayor que	>

(continúa en la siguiente página)

(continúa de la página anterior)

Token	Significado	Regex
TOKEN_LT	Menor que	<
TOKEN_DIFF	Diferente	!=
TOKEN_ISEQUAL	Comparación de igualdad	==

2.1. Espacios en Blanco y Comentarios

Los espacios en blanco y comentarios son reconocidos pero *descartados* por el analizador léxico (no generan token):

Cuadro 2: Patrones descartados

Patrón	Regex	Nota
ESPACIO	<code>[\t\n\r]+</code>	Se ignora
COMENTARIO	<code>//[^\n]*</code>	Comentario de línea
COMENTARIO	<code>/*[\s\S]*?*/</code>	Comentario de bloque

3. Gramática Libre de Contexto

- El conjunto de no terminales es:

$$V = \{ PROGRAMA, VARS, FUNCS, CUERPO, TIPO, ESTATUTO, EXPRESION, EXP, LLAMADA, TERMINO, FACTOR, IMPRIME, CICLO, ASIGNA, CONDICION, CTE \}$$

- El conjunto de terminales es:

$$\Sigma = \{ TOKEN_PROGRAM, TOKEN_START, TOKEN_END, TOKEN_VARS, TOKEN_INT, TOKEN_FLOAT, TOKEN_PRINT, TOKEN_WHILE, TOKEN_DO, TOKEN_IF, TOKEN_ELSE, TOKEN_VOID, TOKEN_RETURN, TOKEN_ID, TOKEN_LETRERO, TOKEN_CTE_INT, TOKEN_CTE_FLOAT, TOKEN_COLON, TOKEN_SEMICOLON, TOKEN_COMMA, TOKEN_LBRACE, TOKEN_RBRACE, TOKEN_LPAREN, TOKEN_RPAREN, TOKEN_SUM, TOKEN_SUBS, TOKEN_TIMES, TOKEN_DIV, TOKEN_EQUALS, TOKEN_GT, TOKEN_LT, TOKEN_DIFF, TOKEN_ISEQUAL \}$$

- P es el conjunto de *producciones*
- $S = PROGRAMA$ es el *símbolo inicial*.

3.1. Producciones

$$CTE \longrightarrow \text{TOKEN_CTE_INT} \mid \text{TOKEN_CTE_FLOAT} \quad (1)$$

$$TIPO \longrightarrow \text{TOKEN_INT} \mid \text{TOKEN_FLOAT} \quad (2)$$

$$\begin{aligned} FACTOR &\longrightarrow \text{TOKEN_LPAREN} \textit{EXPRESION} \text{TOKEN_RPAREN} \\ &\mid \text{TOKEN_SUM} \textit{FACTOR}' \\ &\mid \text{TOKEN_SUBS} \textit{FACTOR}' \\ &\mid \textit{FACTOR}' \\ &\mid \textit{LLAMADA} \end{aligned} \quad (3)$$

$$\begin{aligned} \textit{FACTOR}' &\longrightarrow \textit{CTE} \\ &\mid \text{TOKEN_ID} \end{aligned} \quad (4)$$

$$\textit{LLAMADA} \longrightarrow \text{TOKEN_ID} \text{TOKEN_LPAREN} \textit{LLAMADA}' \text{TOKEN_RPAREN} \quad (5)$$

$$\begin{aligned} \textit{LLAMADA}' &\longrightarrow \textit{EXPRESION} \textit{LLAMADA}'' \\ &\mid \varepsilon \end{aligned} \quad (6)$$

$$\begin{aligned} \textit{LLAMADA}'' &\longrightarrow \text{TOKEN_COMMA} \textit{EXPRESION} \textit{LLAMADA}'' \\ &\mid \varepsilon \end{aligned} \quad (7)$$

$$\textit{TERMINO} \longrightarrow \textit{FACTOR} \textit{TERMINO}' \quad (8)$$

$$\begin{aligned} \textit{TERMINO}' &\longrightarrow \text{TOKEN_TIMES} \textit{TERMINO} \\ &\mid \text{TOKEN_DIV} \textit{TERMINO} \\ &\mid \varepsilon \end{aligned} \quad (9)$$

$$\textit{EXP} \longrightarrow \textit{TERMINO} \textit{EXP}' \quad (10)$$

$$\begin{aligned} \textit{EXP}' &\longrightarrow \text{TOKEN_SUM} \textit{EXP} \\ &\mid \text{TOKEN_SUBS} \textit{EXP} \\ &\mid \varepsilon \end{aligned} \quad (11)$$

$$\textit{EXPRESION} \longrightarrow \textit{EXP} \textit{EXPRESION}' \quad (12)$$

$$\begin{aligned} \textit{EXPRESION}' &\longrightarrow \textit{EXPRESION}'' \\ &\mid \varepsilon \end{aligned} \quad (13)$$

$$\begin{aligned} \textit{EXPRESION}'' &\longrightarrow \text{TOKEN_GT} \textit{EXP} \\ &\mid \text{TOKEN_LT} \textit{EXP} \\ &\mid \text{TOKEN_DIFF} \textit{EXP} \\ &\mid \text{TOKEN_ISEQUAL} \textit{EXP} \end{aligned} \quad (14)$$

$$ASIGNA \rightarrow \text{TOKEN_ID } \text{TOKEN_EQUALS } \textit{EXPRESION} \text{ TOKEN_SEMICOLON} \quad (15)$$

$$\textit{IMPRIME} \rightarrow \text{TOKEN_PRINT } \text{TOKEN_LPAREN } \textit{IMPRIME}' \text{ TOKEN_RPAREN } \text{TOKEN_SEMICOLON} \quad (16)$$

$$\begin{aligned} \textit{IMPRIME}' &\rightarrow \textit{EXPRESION } \textit{IMPRIME}'' \\ &| \text{TOKEN_LETRERO } \textit{IMPRIME}'' \end{aligned} \quad (17)$$

$$\begin{aligned} \textit{IMPRIME}'' &\rightarrow \text{TOKEN_COMMA } \textit{IMPRIME}' \\ &| \varepsilon \end{aligned} \quad (18)$$

$$\textit{CONDICION} \rightarrow \text{TOKEN_IF } \text{TOKEN_LPAREN } \textit{EXPRESION} \text{ TOKEN_RPAREN } \textit{CUERPO } \textit{CONDICION}' \text{ TOKEN_SEMICOLON} \quad (19)$$

$$\begin{aligned} \textit{CONDICION}' &\rightarrow | \text{TOKEN_ELSE } \textit{CUERPO} \\ &| \varepsilon \end{aligned} \quad (20)$$

$$\begin{aligned} \textit{CICLO} &\rightarrow \text{TOKEN_WHILE } \text{TOKEN_LPAREN } \textit{EXPRESION} \text{ TOKEN_RPAREN } \\ &\quad \text{TOKEN_DO } \textit{CUERPO} \text{ TOKEN_SEMICOLON} \end{aligned} \quad (21)$$

$$\begin{aligned} \textit{ESTATUTO} &\rightarrow \textit{ASIGNA} \\ &| \textit{CONDICION} \\ &| \textit{CICLO} \\ &| \textit{IMPRIME} \\ &| \textit{LLAMADA} \text{ TOKEN_SEMICOLON} \\ &| \text{TOKEN_LBRACKET } \textit{ESTATUTO}' \text{ TOKEN_RBRACKET} \end{aligned} \quad (22)$$

$$\begin{aligned} \textit{ESTATUTO}' &\rightarrow \textit{ESTATUTO } \textit{ESTATUTO}' \\ &| \varepsilon \end{aligned} \quad (23)$$

$$\textit{VARS} \rightarrow \text{TOKEN_VARS } \textit{VARS}' \quad (24)$$

$$\textit{VARS}' \rightarrow \text{TOKEN_ID } \textit{VARS}'' \text{ TIPO } \text{TOKEN_SEMICOLON } \textit{VARS}''' \quad (25)$$

$$\begin{aligned} \textit{VARS}'' &\rightarrow \text{TOKEN_COMMA } \text{TOKEN_ID } \textit{VARS}'' \\ &| \text{TOKEN_COLON} \end{aligned} \quad (26)$$

$$\begin{aligned} \textit{VARS}''' &\rightarrow \textit{VARS}' \\ &| \varepsilon \end{aligned} \quad (27)$$

$$\textit{CUERPO} \rightarrow \text{TOKEN_LBRACE } \textit{CUERPO}' \text{ TOKEN_RBRACE} \quad (28)$$

$$\begin{aligned} \textit{CUERPO}' &\rightarrow \textit{ESTATUTO } \textit{CUERPO}' \\ &| \varepsilon \end{aligned} \quad (29)$$

$$\begin{aligned} \textit{FUNCS} &\rightarrow \text{TOKEN_VOID } \text{TOKEN_ID } \text{TOKEN_LPAREN } \textit{FUNC}' \text{ TOKEN_RPAREN} \\ &\quad \text{TOKEN_LBRACE } \textit{FUNC}''' \textit{CUERPO } \textit{FUNCS}''' \text{ TOKEN_RBRACE } \text{TOKEN_SEMICOLON} \\ &| \text{TIPO } \text{TOKEN_ID } \text{TOKEN_LPAREN } \textit{FUNC}' \text{ TOKEN_RPAREN} \\ &\quad \text{TOKEN_LBRACE } \textit{FUNC}''' \textit{CUERPO } \textit{FUNCS}''' \text{ TOKEN_RBRACE } \text{TOKEN_SEMICOLON} \end{aligned} \quad (30)$$

$$\begin{aligned} \textit{FUNCS}' &\rightarrow \text{TOKEN_ID } \text{TOKEN_COLON } \textit{TIPO } \textit{FUNC}'' \\ &| \varepsilon \end{aligned} \quad (31)$$

$$\begin{aligned} FUNCS'' &\rightarrow \text{TOKEN_COMMA } \text{TOKEN_ID } \text{TOKEN_COLON } \textit{TIPO } FUNCS'' \\ &\quad | \quad \varepsilon \end{aligned} \quad (32)$$

$$\begin{aligned} FUNCS''' &\rightarrow \text{TOKEN_VARS} \\ &\quad | \quad \varepsilon \end{aligned} \quad (33)$$

$$\begin{aligned} FUNCS'''' &\rightarrow \text{TOKEN_RETURN } \textit{EXPRESION } \text{TOKEN_SEMICOLON} \\ &\quad | \quad \varepsilon \end{aligned} \quad (34)$$

$$\begin{aligned} \textit{PROGRAMA} &\rightarrow \text{TOKEN_PROGRAM } \text{TOKEN_ID } \text{TOKEN_SEMICOLON } \textit{PROGRAMA}' \textit{PROGRAMA}'' \\ &\quad \text{TOKEN_START } \textit{CUERPO } \text{TOKEN_END} \end{aligned} \quad (35)$$

$$\begin{aligned} \textit{PROGRAMA}' &\rightarrow \textit{VARS} \\ &\quad | \quad \varepsilon \end{aligned} \quad (36)$$

$$\begin{aligned} \textit{PROGRAMA}'' &\rightarrow \textit{FUNCS } \textit{FPROGRAMA}'' \\ &\quad | \quad \varepsilon \end{aligned} \quad (37)$$

4. Léxico y Sintaxis

Como parte de esta sección llevé a cabo una investigación de distintas herramientas automáticas de Scanners y Parsers.

A continuación se presentan los resultados de esta investigación.

Cuadro 3: Análisis de Yacc

Lex & Yacc	
Evaluación	Nota
Plataforma y lenguaje base	C, cross platform
Formato requerido en las entradas	Hay dos declaraciones, en C y en LEX. Las importantes son en C y en LEX se definen regex y simbolos
Tipo de ejecución	YACC crea el parser, y crea los tokens del LEX después lo traslada a C
Tipo de licenciamiento	MIT License, gratis
Herramientas teóricas en las quese basan	Expresiones regulares y gramática libre de contexto

Cuadro 4: Análisis de Lark

Lark	
Evaluación	Nota
Plataforma y lenguaje base	python
Formato requerido en las entradas	Utiliza string de Python y archivos .lark
Tipo de ejecución	Construye un árbol en base a la gramática AST, ejecución embebida
Tipo de licenciamiento	MIT License, gratis
Herramientas teóricas en las que se basan	Expresiones regulares, LARL EARLEY y CYK

Cuadro 5: Análisis de LALRPOP

LALRPOP	
Evaluación	Nota
Plataforma y lenguaje base	rust
Formato requerido en las entradas	Estilo YACC
Tipo de ejecución	Construye un árbol en base a la gramática AST
Tipo de licenciamiento	MIT License, gratis
Herramientas teóricas en las que se basan	Expresiones regulares, gramáticas libres de contexto

Tras esta investigación, se optó por utilizar lark como parser y scanner para la continuidad del proyecto. Los beneficios principales de esta herramienta se verán reflejados en el tiempo de desarrollo del proyecto, así como la facilidad de uso. Se busca obtener un parser funcional lo más pronto posible, y Lark es una herramienta que se adapta a esta necesidad.

4.1. Configuración de Lark

Para la elaboración del proyecto creamos el módulo de python `patito.py`. En este módulo se importó la librería de Lark, y se creó un parser utilizando la gramática definida en la sección 3. A continuación se presenta el string con las llaves - valor que procesan la gramática libre de contexto definida en la sección 3, además se utilizó `common.ESCAPED_STRING` para el token de letrero, y se ignoraron los espacios en blanco utilizando `common.WS`. En Mayúscula se encuentran los tokens, y en minúscula las producciones de la gramática.

```

patito_grammar = r"""
    TOKEN_PROGRAM:      "PROGRAM"
    TOKEN_START:         "BEGIN"
    TOKEN_END:           "END"
    TOKEN_VARS:          "VARS"
    TOKEN_INT:           "INT"
    TOKEN_FLOAT:         "FLOAT"
    TOKEN_PRINT:         "PRINT"
    TOKEN_WHILE:         "WHILE"
    TOKEN_DO:            "DO"
    TOKEN_IF:            "IF"
    TOKEN_ELSE:          "ELSE"
    TOKEN_VOID:          "VOID"
    TOKEN_RETURN:        "RETURN"
    TOKEN_ID:            /[a-zA-Z\_][a-zA-Z0-9\_]*\/
    TOKEN_LETRERO:       ESCAPED_STRING
    TOKEN_CTE_INT:       /[0-9]+\/
    TOKEN_CTE_FLOAT:     /[0-9]+\.[0-9]+\/
    TOKEN_SEMICOLON:     ";"
    TOKEN_COMMA:         ","
    TOKEN_COLON:         ":"
    TOKEN_LBRACE:        "{"
    TOKEN_RBRACE:        "}"
    TOKEN_LBRACKET:      "["
    TOKEN_RBRACKET:      "]"
    TOKEN_LPAREN:        "("
    TOKEN_RPAREN:        ")"
    TOKEN_SUM:           "+"
    TOKEN_SUBS:          "-"
    TOKEN_DIV:           "/"
    TOKEN_TIMES:         "*"
    TOKEN_EQUALS:        "="
    TOKEN_GT:            ">"
    TOKEN_LT:            "<"
    TOKEN_DIFF:          "!="
    TOKEN_ISEQUAL:       "=="

    cte:                  TOKEN_CTE_FLOAT
                        | TOKEN_CTE_INT

    tipo:                 TOKEN_INT
                        | TOKEN_FLOAT

    factor:               TOKEN_LPAREN expresion TOKEN_RPAREN
                        | TOKEN_SUM factor_
                        | TOKEN_SUBS factor_
                        | factor_
                        | llamada

    factor_:              cte

```

```

                                | TOKEN_ID
llamada:                       TOKEN_ID TOKEN_LPAREN llamada_ TOKEN_RPAREN
llamada_:                      expresion llamada__
                                |
llamada__:                     TOKEN_COMMA expresion llamada__
                                |
termino :                      factor termino_
termino_:                     TOKEN_TIMES termino
                                | TOKEN_DIV termino
                                |
exp:                           termino exp_
exp_:                         TOKEN_SUM exp
                                | TOKEN_SUBS exp
                                |
expresion:                     exp expresion_
expresion_:                   expresion__
                                |
expresion__:                  TOKEN_GT exp
                                | TOKEN_LT exp
                                | TOKEN_DIFF exp
                                | TOKEN_ISEQUAL exp

asigna:                       TOKEN_ID TOKEN_EQUALS expresion TOKEN_SEMICOLON
imprime:                      TOKEN_PRINT TOKEN_LPAREN imprime_ TOKEN_RPAREN TOKEN_SEMICOLON
imprime_:                     expresion imprime__
                                | TOKEN_LETRERO imprime__
imprime__:                    TOKEN_COMMA imprime_
                                |
condicion:                    TOKEN_IF TOKEN_LPAREN expresion TOKEN_RPAREN cuerpo condicion_ TOKEN_S
condicion_:                   TOKEN_ELSE cuerpo
                                |
ciclo:                        TOKEN_WHILE TOKEN_LPAREN expresion TOKEN_RPAREN TOKEN_DO cuerpo TOKEN_
estatuto:                     asigna
                                | condicion
                                | ciclo
                                | imprime
                                | llamada TOKEN_SEMICOLON
                                | TOKEN_LBRACKET estatuto_ TOKEN_RBRACKET
estatuto_:                    estatuto estatuto_
                                |
vars:                         TOKEN_VARS vars_
vars_:                        TOKEN_ID vars__ tipo TOKEN_SEMICOLON vars___
vars__:                       TOKEN_COMMA TOKEN_ID vars__
                                | TOKEN_COLON
vars___:                      vars_
                                |
cuerpo:                       TOKEN_LBRACE cuerpo_ TOKEN_RBRACE
cuerpo_:                      estatuto cuerpo_

```

```

funcs:          TOKEN_VOID TOKEN_ID TOKEN_LPAREN funcs_ TOKEN_RPAREN TOKEN_LBRACE func
| tipo TOKEN_ID TOKEN_LPAREN funcs_ TOKEN_RPAREN TOKEN_LBRACE funcs__
funcs_:         TOKEN_ID TOKEN_COLON tipo funcs__
|
funcs__:        TOKEN_COMMA TOKEN_ID TOKEN_COLON tipo funcs__
|
funcs___:       vars
|
funcs____:      TOKEN_RETURN expresion TOKEN_SEMICOLON
|
programa:       TOKEN_PROGRAM" "TOKEN_ID TOKEN_SEMICOLON programa_ programa__ TOKEN_ST
programa_:      vars
|
programa__:     funcs programa__
|

%import common.WS
%ignore WS
%import common.ESCAPED_STRING
"""

```

5. Casos de prueba

Se realizaron 10 pruebas que exploran distintas partes de la gramática, y se verificó que el parser de Lark las procesara correctamente. A continuación se presentan los casos de prueba realizados.

```

"""
% =====

"name" : "TC 1. Programa vacio",
"input": "PROGRAM A ; BEGIN { } END",

"description": "Un programa mínimo con el nombre A, sin variables
ni funciones, y un bloque vacío.",

"should_parse": True

% =====

"name" : "TC 2. Programa con typo",
"input": "PROGRAMA A ; BEGIN { } END",

"description": "Un programa con un error tipográfico en la palabra clave
'PROGRAM', lo que debería causar un error de análisis sintáctico.",

```

```
"should_parse": False
```

```
% =====
```

```
"name" : "TC 3. Programa con VARS",
```

```
"input": '''
```

```
    PROGRAM A ;
```

```
    VARS
```

```
    a, b, c : INT;
```

```
    d,e,f: FLOAT;
```

```
    BEGIN { } END
```

```
''',
```

```
"description": "Un programa que declara variables enteras y flotantes  
utilizando la sección VARS, pero sin funciones ni código en el  
bloque principal.",
```

```
"should_parse": True
```

```
% =====
```

```
"name" : "TC 4. Programa con VARS y FUNCS",
```

```
"input": '''
```

```
    PROGRAM A ;
```

```
    VARS
```

```
    a, b, c : INT;
```

```
    d,e,f: FLOAT;
```

```
    VOID main(aa : INT, bb: FLOAT) {
```

```
        VARS
```

```
        ab : FLOAT;
```

```
        {
```

```
            ab = aa + bb;
```

```
        }
```

```
    };
```

```
    BEGIN { } END
```

```
''',
```

```
"description": "Un programa que incluye tanto variables globales  
como una función principal con su propia sección de variables]  
locales y un bloque de código que realiza una operación simple.",
```

```
"should_parse": True
```

```
% =====
```

```
"name" : "TC 5. Programa con FUNCS antes que VARS",
```

```
"input": '''
```

```
    PROGRAM A ;
```

```

        VOID main(aa : INT, bb: FLOAT) {
            VARS
            ab : FLOAT;
            {
                ab = aa + bb;
            }
        };
        VARS
        a, b, c : INT;
        d,e,f: FLOAT;
        BEGIN { } END
    ''',

```

"description": "Un programa que declara una función antes de la sección de variables globales, lo que debería causar un error de análisis sintáctico debido a la violación del orden esperado de las secciones.",

"should_parse": False

% =====

"name" : "TC 6. Programa con FUNCS multiples estatutos",

"input": '''

```

        PROGRAM A ;
        VARS
        a, b, c : INT;
        d,e,f: FLOAT;
        VOID main(aa : INT, bb: FLOAT) {
            VARS
            ab : FLOAT;
            {
                ab = aa + bb;
                PRINT("a");
            }

        };

```

```

        BEGIN { } END
    ''',

```

"description": "Un programa que incluye una función principal con múltiples declaraciones y un bloque de código que realiza varias operaciones, incluyendo una asignación y una llamada a impresión.",

"should_parse": True

% =====

```

"name" : "TC 7. Programa con if y while",
"input": '''
    PROGRAM A ;
    VARS
    a, b, c : INT;
    d,e,f: FLOAT;
    VOID main(aa : INT, bb: FLOAT) {
        VARS
        ab : FLOAT;
        {
            ab = aa + bb;
            PRINT("a");
        }

    };

    BEGIN {
        WHILE(a > b) DO {
            IF(a > c){
                PRINT("a es mayor que c");
            } ELSE {
                PRINT("a no es mayor que c");
            };
        };
    } END
'''
"description": "Un programa que incluye una función
principal con un bloque de código que contiene una estructura
de control WHILE anidada con una estructura IF-ELSE",

"should_parse": True

% =====

"name" : "TC 8. Programa con while roto",
"input": '''
    PROGRAM A ;
    VARS
    a, b, c : INT;
    d,e,f: FLOAT;
    VOID main(aa : INT, bb: FLOAT) {
        VARS
        ab : FLOAT;
        {
            ab = aa + bb;
            PRINT("a");
        }
    }
'''

```

```

};

BEGIN {
    WHILE(a > b) DO
        IF(a > c){
            PRINT("a es mayor que c");
        } ELSE {
            PRINT("a no es mayor que c");
        };
    };
} END
''',

"description": "Un programa que contiene una estructura WHILE
sin llaves que envuelvan su bloque de código, lo que debería
causar un error de análisis sintáctico debido a la falta de
delimitadores adecuados para el bloque del WHILE.",

"should_parse": False

% =====

"name" : "TC 9. Programa con if roto",
"input": '''
    PROGRAM A ;
    VARS
    a, b, c : INT;
    d,e,f: FLOAT;
    VOID main(aa : INT, bb: FLOAT) {
        VARS
        ab : FLOAT;
        {
            ab = aa + bb;
            PRINT("a");
        }

    };

    BEGIN {
        WHILE(a > b) DO {
            IF(a > c){
                PRINT("a es mayor que c");
            } ELSE
                PRINT("a no es mayor que c");
            };
        };
    } END

```

```

    ''',

    "description": "Un programa que contiene una estructura IF sin
    llaves que envuelvan su bloque ELSE, lo que debería causar un
    error de análisis sintáctico debido a la falta de delimitadores
    adecuados para el bloque del ELSE.",

    "should_parse": False

% =====

    "name" : "TC 10. Estatutos vacios",
    "input": '''
        PROGRAM A ;
        VARS
        a, b, c : INT;
        d,e,f: FLOAT;
        VOID main(aa : INT, bb: FLOAT) {
            VARS
            ab : FLOAT;
            {
                ab = aa + bb;
                PRINT("a");
            }

        };

        BEGIN {
            WHILE(a > b) DO {
                IF(a > c){
                } ELSE {
                };
            };
        } END
    '''

    "description": "Un programa que incluye estructuras de control IF
    y WHILE con bloques vacíos, lo que debería ser válido sintácticamente
    aunque no realice ninguna operación dentro de esos bloques.",

    "should_parse": True

% =====

    "name" : "TC11. LLamada",
    "input": '''
        PROGRAM A ;
        VARS

```

```

a, b, c : INT;
d,e,f: FLOAT;
VOID main(aa : INT, bb: FLOAT) {
    VARS
    ab : FLOAT;
    {
        ab = aa + bb;
        PRINT("a");
    }

};

BEGIN {
    WHILE(a > b) DO {
        IF(a > c){
            PRINT("a es mayor que c");
        } ELSE {
            funcion_test(1,2);
        }
    };
} END
'''
"description": "Un programa que incluye una llamada a
función dentro de un bloque ELSE, lo que debería ser válido",
"should_parse": True

```

"""

Todos los casos de prueba fueron procesados correctamente por el parser de Lark, obteniendo los resultados esperados definidos en el campo `should_parse`.

6. Cubo Semántico

A continuación se presenta el cubo semántico para las operaciones aritméticas y relacionales de los tipos de datos `INT` y `FLOAT`, así como el resultado de las operaciones relacionales que siempre es de tipo `BOOL`. Cualquier operación no definida en el cubo semántico se considera un error semántico.

Cuadro 6: Cubo semántico de Patito

Op1	Op2	Operador	Resultado
INT	INT	+	INT
INT	FLOAT	+	FLOAT
FLOAT	INT	+	FLOAT
FLOAT	FLOAT	+	FLOAT
INT	INT	-	INT
INT	FLOAT	-	FLOAT
FLOAT	INT	-	FLOAT
FLOAT	FLOAT	-	FLOAT
INT	INT	*	INT
INT	FLOAT	*	FLOAT
FLOAT	INT	*	FLOAT
FLOAT	FLOAT	*	FLOAT
INT	INT	/	FLOAT
INT	FLOAT	/	FLOAT
FLOAT	INT	/	FLOAT
FLOAT	FLOAT	/	FLOAT
INT	INT	><!= ==	BOOL
INT	FLOAT	><!= ==	BOOL
FLOAT	INT	><!= ==	BOOL
FLOAT	FLOAT	><!= ==	BOOL
INT	INT	cualquier otro	ERROR
FLOAT	FLOAT	cualquier otro	ERROR

7. Estructuras de Datos

7.1. Directorio de Funciones

Se definió la siguiente estructura para el Directorio de Funciones, donde cada función tiene un nombre, un tipo de retorno, y una referencia a su Tabla de Variables local. La función principal del programa se incluye como una entrada en el Directorio de Funciones con un scope global.

Cuadro 7: Directorio de Funciones

Nombre	Tipo retorno	Tabla de variables
programa	void	$\rightarrow$ TVar _{global}
func1	void	$\rightarrow$ TVar _{func1}
...	...	...

7.2. Tabla de Variables

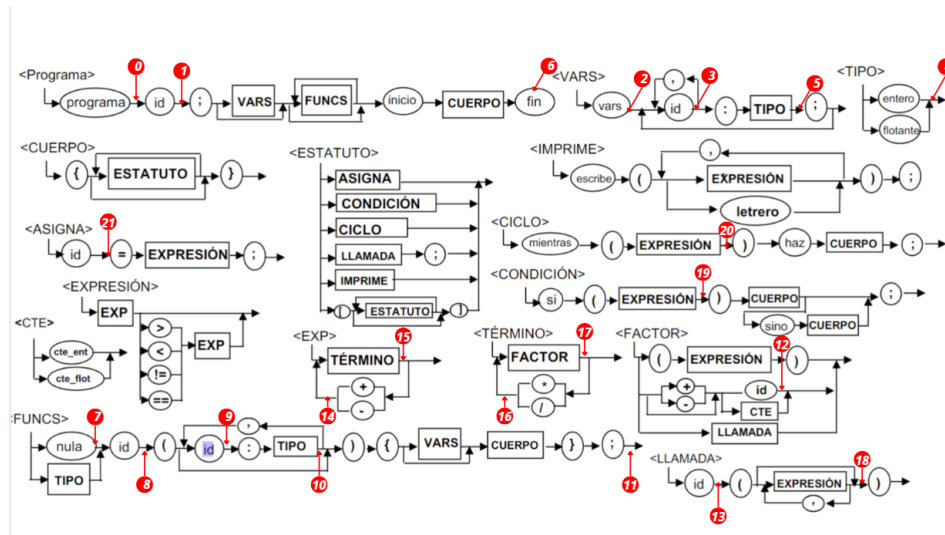
Por otra parte, la Tabla de Variables se estructura con el nombre de la variable, categoría, su tipo de dato, y el scope al que pertenece (global o local a una función). Cada función tiene su propia Tabla de Variables local, y las variables globales se almacenan en una Tabla de Variables global accesible desde cualquier función.

Cuadro 8: Tabla de Variables

Nombre	Categoría	Tipo	Scope
x	VAR	FLOAT	global
y	PARAM	INT	local
z	VAR	INT	local
...	...	...	...

8. Puntos Neurálgicos Semánticos

Se presenta el mapa de puntos neurálgicos del compilador, donde se identifican las producciones clave de la gramática que requieren acciones semánticas específicas para la construcción de las estructuras de datos y la validación de tipos. Para cada punto se describe la acción a realizar y la validación correspondiente para asegurar la corrección semántica del programa.



Cuadro 9: Puntos neurálgicos del compilador

Punto	Acción	Validación
P0	Crear entrada en DirFun	—
P1	Define el nombre del programa y el scope como global	—

Punto	Acción	Validación
P2	Si la función actual aún no tiene VarTable la crea y la enlaza a la función con scope actual	—
P3	Verifica que el <code>TOKEN_ID</code> no exista en VarTable del scope actual <code>current_id = TOKEN_ID</code>	Error: Variable doblemente declarada
P4	<code>current_tipo = TOKEN_TIPO</code>	—
P5	Agrega [<code>current_id</code> , <code>VAR</code> , <code>current_tipo</code> , <code>global</code>] a la VarTable por cada variable declarada	—
P6	Cierra scope global y elimina la de VarTable local de memoria	—
P7	<code>current_tipo = TOKEN_VOID</code>	—
P8	Verifica que el <code>TOKEN_ID</code> no exista en el DirFun si no existe: se agrega al DirFun con [<code>TOKEN_ID</code> , <code>current_tipo</code>] Si existe: Error	Error: Función doblemente declarada
P9	Si no existe una, crea nueva VarTable local y la asocia a la función. Revisa si el <code>TOKEN_ID</code> del parametro existe en la VarTable si no: <code>current_id = TOKEN_ID</code> si existe: Error	Error: parametro doblemente declarada
P10	Crea un registro en la VarTable con [<code>current_id</code> , <code>PARAM</code> , <code>current_tipo</code> , <code>local</code>]	—
P11	Cierra el scope local y elimina VarTable local de memoria	—
P12	Verifica que <code>TOKEN_ID</code> exista en VarTable (scope local o global)	Error: Variable no declarada
P13	Verifica que <code>TOKEN_ID</code> exista en DirFun	Error: Función no declarada
P14	Se agrega el operador a POper	—

Punto	Acción	Validación
P15	Si hay un operando (+ o -) en POper se consultas el cubo semántico con tipos de operandos y operador	Error: Operación entre tipos incompatibles
P16	Se agrega el operador a POper	—
P17	Si hay un operando (* o /) en POper se consultas el cubo semántico con tipos de operandos y operador	Error: Operación entre tipos incompatibles
P18	Verifica que los parametros tengan los tipos correctos para la función llamada	Error: Tipo de parametro inesperado
P19	Verifica que el tipo de EXPRESION sea BOOL	Error: La condición debe ser de tipo booleano
P20	Verifica que el tipo de EXPRESION sea BOOL	Error: La condición del ciclo debe ser de tipo booleano
P21	Verifica que el tipo de la asignación sea compatible con el tipo de la variable	Error: Tipo incompatible
P22	Al encontrar TOKEN_RETURN: evalúa EXPRESION y registra su tipo en DirFun. Al cerrar el scope, verifica que el tipo registrado coincida con el tipo de retorno declarado de la función. Las funciones VOID no pueden contener RETURN.	Error: Tipo de retorno incorrecto; función VOID con RETURN; función no-VOID sin RETURN

8.1. Justificación

Se eligió la tabla hash para nuestras estructuras de datos ya que las consultas en esta estructura son de costo constante ($O(1)$).

8.2. Operaciones principales

Cuadro 10: Operaciones sobre el Cubo Semántico

Operación	Costo	Descripción
<code>consultar(t1, t2, op)</code>	$O(1)$	Retorna el tipo resultado de aplicar <code>op</code> entre <code>t1</code> y <code>t2</code>
<code>esError(resultado)</code>	$O(1)$	Verifica si el resultado es ERROR para lanzar error de tipo incompatible
<code>operacionValida(t1, t2, op)</code>	$O(1)$	Retorna True si la combinación de tipos y operador existe en el cubo

Cuadro 11: Operaciones sobre DirFun

Operación	Costo	Descripción
<code>agregar(nombre, tipo)</code>	$O(1)$	Registra una nueva función con su tipo de retorno e inicializa su tabla de variables vacía
<code>existe(nombre)</code>	$O(1)$	Verifica si una función ya fue declarada. Detectar redeclaraciones
<code>tipoRetorno(nombre)</code>	$O(1)$	Retorna el tipo de retorno de la función
<code>getTablaVars(nombre)</code>	$O(1)$	Retorna la tabla de variables asociada a la función

Cuadro 12: Operaciones sobre VarTable

Operación	Costo	Descripción
agregar(nombre, cat, tipo, scope)	$O(1)$	Registra una variable con su categoría, tipo y scope correspondiente
existe(nombre)	$O(1)$	Verifica si una variable ya fue declarada. Para detectar redeclaraciones
tipo(nombre)	$O(1)$	Retorna el tipo de la variable usado como entrada al Cubo Semántico
varsEnScope(scope)	$O(n)$	Retorna todas las variables que pertenecen al scope indicado
limpiarScope(scope)	$O(n)$	Elimina todas las variables del scope indicado al salir de una función

8.3. Definición en código

Se agregan las definiciones de los elementos definidos anteriormente.

8.3.1. Cubo Semántico

```
CUBO_SEMANTICO = {
  ("INT", "INT"): {"+": "INT", "-": "INT", "*": "INT", "/": "FLOAT",
                  ">": "BOOL", "<": "BOOL", "!=": "BOOL", "==" : "BOOL"},
  ("INT", "FLOAT"): {"+": "FLOAT", "-": "FLOAT", "*": "FLOAT", "/": "FLOAT",
                    ">": "BOOL", "<": "BOOL", "!=": "BOOL", "==" : "BOOL"},
  ("FLOAT", "INT"): {"+": "FLOAT", "-": "FLOAT", "*": "FLOAT", "/": "FLOAT",
                    ">": "BOOL", "<": "BOOL", "!=": "BOOL", "==" : "BOOL"},
  ("FLOAT", "FLOAT"): {"+": "FLOAT", "-": "FLOAT", "*": "FLOAT", "/": "FLOAT",
                      ">": "BOOL", "<": "BOOL", "!=": "BOOL", "==" : "BOOL"},
}
```

8.3.2. DirFun

```
DirFun {
  nombre_func -> {
    tipo_retorno : "VOID",
    tabla_vars   : TablaVariables
  }
}
```

8.3.3. VarTable

```
VarTable {
  nombre -> {
```

```

        categoria : "VAR" | "PARAM",
        tipo      : "INT" | "FLOAT",
        scope     : "global" | nombre_funcion
    }
}

```

9. Generación de Cuádruplos

Esta sección documenta la tercera etapa del compilador Patito la traducción del árbol sintáctico a cuádruplos. Se implementa en el módulo `patito_cuadрупlos.py`.

9.1. Estructuras de Datos

9.1.1. Pilas

La clase `Pila` es una estructura LIFO genérica. Durante la generación de cuádruplos se instancian tres pilas independientes

- `pila_ops` almacena operadores aritméticos y relacionales pendientes de procesar.
- `pila_and`s almacena operandos (nombres de variables, constantes y temporales).
- `pila_tipos` almacena el tipo semántico (`INT`, `FLOAT`, `BOOL`) de cada operando en `pila_and`s.

9.1.2. Fila de Cuádruplos (`FilaCuadрупlos`)

La clase `FilaCuadрупlos` es una estructura FIFO que almacena la secuencia de cuádruplos generados. Cada cuádruplo tiene la forma:

$$(op, arg1, arg2, resultado)$$

Cuadro 13: Operaciones de `FilaCuadрупlos`

Operación	Costo	Descripción
<code>agregar(op, a1, a2, res)</code>	$O(1)$	Inserta el cuádruplo al final de la fila y retorna su índice
<code>completar(idx, campo, val)</code>	$O(1)$	Rellena un campo pendiente (<code>_</code>) en el cuádruplo de índice <code>idx</code>
<code>siguiente()</code>	$O(1)$	Retorna el índice del próximo cuádruplo a generar

9.2. Puntos Neurálgicos del Generador de Cuádruplos

A continuación se presenta cada producción gramatical relevante con sus puntos neurálgicos anotados entre corchetes `[P_XXX]`.

Operandos

```
factor_:  cte                [P_PUSH_OPERAND]
         | TOKEN_ID         [P_PUSH_OPERAND]
```

[P_PUSH_OPERAND] Ahrega el valor del operando a `pila_and`s y su tipo semántico (INT o FLOAT) a `pila_tipos`. Para `TOKEN_ID` se consulta el directorio de funciones para obtener el tipo.

Multiplicación y División

```
termino_: TOKEN_TIMES termino [P_PUSH_OP_MUL -> P_GEN]
         | TOKEN_DIV  termino [P_PUSH_OP_MUL -> P_GEN]
         | <empty>
```

[P_PUSH_OP_MUL] — Empuja `*` o `/` a `pila_ops`, evalúa el `termino` derecho y luego dispara [P_GEN]: extrae operador, dos operandos y sus tipos, consulta el cubo semántico, genera el cuádruplo (`op`, `izq`, `der`, `tn`) y empuja el temporal resultado.

Suma y Resta

```
exp_: TOKEN_SUM  exp [P_PUSH_OP_SUM -> P_GEN]
     | TOKEN_SUBS exp [P_PUSH_OP_SUM -> P_GEN]
     | <empty>
```

[P_PUSH_OP_SUM] — Empuja `+` o `-` a `pila_ops`, evalúa el `exp` derecho y dispara [P_GEN].

Operadores Relacionales

```
expresion__: TOKEN_GT      exp [P_PUSH_OP_REL -> P_GEN]
             | TOKEN_LT      exp [P_PUSH_OP_REL -> P_GEN]
             | TOKEN_DIFF     exp [P_PUSH_OP_REL -> P_GEN]
             | TOKEN_ISEQUAL exp [P_PUSH_OP_REL -> P_GEN]
```

[P_PUSH_OP_REL] Agrega el operador relacional a `pila_ops`, evalúa el `exp` derecho y dispara [P_GEN], produciendo un temporal de tipo `BOOL`.

Asignación

```
asigna: TOKEN_ID TOKEN_EQUALS expresion TOKEN_SEMICOLON [P_ASIGNA]
```

[P_ASIGNA] Evalúa `expresion`, extrae el resultado de `pila_and`s y genera el cuádruplo (`=`, `resultado`, `_`, `var`).

Impresión

```
imprime_: expresion imprime__ [P_PRINT]
         | TOKEN_LETRERO imprime__ [P_PRINT_LETRERO]
```

[P_PRINT] Evalúa la expresión, extrae el resultado y genera (`PRINT`, `resultado`, `_`, `_`).
[P_PRINT_LETRERO] Genera directamente (`PRINT`, `"letrero"`, `_`, `_`) sin evaluar expresión.

Condicional

condicion:

TOKEN_IF	TOKEN_LPAREN	
expresion		[P_IF_COND]
TOKEN_RPAREN		[P_GOTO?]
cuerpo		[P_IF_BODY]
condicion_		[P_GOTO? / P_FILL_IF]
TOKEN_SEMICOLON		

condicion_: TOKEN_ELSE cuerpo [P_ELSE_BODY / P_FILL_ELSE]
| <empty>

- [P_IF_COND] Evalúa la expresión booleana de la condición.
- [P_GOTO?] Genera (GOTO, cond, _, _) con destino pendiente.
- [P_IF_BODY] Genera los cuádruplos del bloque IF.
- [P_GOTO] Si existe ELSE: genera (GOTO, _, _, _) con destino pendiente para saltar el bloque ELSE.
- [P_FILL_IF] Rellena el destino del GOTO con la posición actual.
- [P_ELSE_BODY] Genera los cuádruplos del bloque ELSE.
- [P_FILL_ELSE] Rellena el destino del GOTO con la posición actual.

Ciclo

ciclo:

TOKEN_WHILE		[P_WHILE_START]
TOKEN_LPAREN		
expresion		[P_WHILE_COND]
TOKEN_RPAREN		[P_GOTO?]
TOKEN_DO		
cuerpo		[P_WHILE_BODY]
		[P_GOTO_BACK]
		[P_FILL_WHILE]
TOKEN_SEMICOLON		

- [P_WHILE_START] Guarda el índice actual de la fila como `pos_inicio` (punto de regreso del ciclo).
- [P_WHILE_COND] Evalúa la expresión booleana de la condición del ciclo.
- [P_GOTO?] Genera (GOTO, cond, _, _) con destino pendiente.
- [P_WHILE_BODY] Genera los cuádruplos del cuerpo del ciclo.
- [P_GOTO_BACK] Genera (GOTO, _, _, `pos_inicio`) para regresar al inicio del ciclo.
- [P_FILL_WHILE] Rellena el destino del GOTO con la posición actual (salida del ciclo).

Llamada a función

llamada: TOKEN_ID TOKEN_LPAREN llamada_ TOKEN_RPAREN [P_CALL]

llamada_: expresion llamada__ [P_PARAM]
| <empty>

[P_PARAM] Por cada argumento evaluado genera (PARAM, arg, _, _).

[P_CALL] Verifica que la función exista en DirFun y genera (CALL, nombre_func, _, tn), además verifica que los parametros sean del tipo y cantidad correcta.

[P_RETURN] Verifica que el valor de retorno sea del tipo correcto y genera (RETURN, val, _, _).

9.3. Tabla de Puntos Neurálgicos

Cuadro 14: Puntos neurálgicos del generador de cuádruplos

Punto	Acción	Cuádruplo generado
P_PUSH_OPERAND	Agrega operando y tipo a <code>pila_and</code> s/ <code>pila_tipos</code>	—
P_PUSH_OP_MUL	Agrega * o / a <code>pila_ops</code>	—
P_PUSH_OP_SUM	Agrega + o - a <code>pila_ops</code>	—
P_PUSH_OP_REL	Agrega operador relacional a <code>pila_ops</code>	—
P_GEN	Pop operador + 2 operandos, consulta cubo semántico, genera cuádruplo, empuja temporal	(op, izq, der, tn)
P_ASIGNA	Pop resultado de <code>pila_and</code> s, genera cuádruplo de asignación	(=, val, _, var)
P_PRINT	Pop resultado o letrero, genera cuádruplo de impresión	(PRINT, val, _, _)
P_IF_COND	Evalúa expresión booleana del IF	—
P_GOTOF	Genera salto condicional con destino pendiente	(GOTOF, cond, _, ?)
P_GOTO	Genera salto incondicional con destino pendiente (fin del IF)	(GOTO, _, _, ?)
P_FILL_IF	Rellena destino del GOTOF	—
P_FILL_ELSE	Rellena destino del GOTO	—
P_WHILE_START	Guarda <code>pos_inicio</code> de la fila	—
P_GOTO_BACK	Genera salto de regreso al inicio del ciclo	(GOTO, _, _, pos_inicio)

Punto	Acción	Cuádruplo generado
P_FILL_WHILE	Rellena destino del GOTOF del ciclo	—
P_PARAM	Genera cuádruplo de parámetro para la llamada	(PARAM, arg, _, _)
P_CALL	Verifica existencia de la función. Genera cuádruplo de llamada	(CALL, func, _, tn)
P_RETURN	Verifica que el valor de retorno sea del tipo correcto	(RETURN, val, _, _)

9.4. Resultados de Casos de Prueba

A continuación se presentan los cuádruplos generados por `patito_cuadрупlos.py` para cada caso de prueba.

TEST 1. Expresiones aritméticas

Programa con asignaciones enteras y expresión `a + b * 2`. La multiplicación se resuelve primero (mayor precedencia) produciendo el temporal `t1`, luego la suma produce `t2`.

#	Op	Arg1	Arg2	Resultado

0	=	5	-	a
1	=	3	-	b
2	*	b	2	t1
3	+	a	t1	t2
4	=	t2	-	c
5	PRINT	c	-	-

TEST 2. Flotantes y PRINT con letrero

Programa con variables `FLOAT` y `PRINT` de letrero seguido de expresión. Los literales flotantes `2.0` y `1.0` se reconocen como tipo `FLOAT` en el cubo semántico.

#	Op	Arg1	Arg2	Resultado

0	=	4	-	n
1	=	3.14	-	x
2	*	x	2.0	t1
3	+	t1	1.0	t2
4	=	t2	-	x
5	PRINT	"Resultado: "	-	-
6	PRINT	x	-	-

TEST 3. Condicional IF-ELSE

El cuádruplo `GOTOF` (índice 3) salta al cuádruplo 6 (inicio del `ELSE`) si la condición es falsa. El `GOTO` (índice 5) salta al cuádruplo 7 para omitir el bloque `ELSE` cuando la condición fue verdadera.

#	Op	Arg1	Arg2	Resultado
0	=	10	-	a
1	=	5	-	b
2	>	a	b	t1
3	GOTOF	t1	-	6
4	PRINT	"a es mayor"	-	-
5	GOTO	-	-	7
6	PRINT	"b es mayor o igual"	-	-

TEST 4. Ciclo WHILE

P_WHILE_START guarda `pos_inicio = 2` (cuádruplo de la condición). El GOTOF en el índice 3 salta al cuádruplo 9 (salida del ciclo) cuando `i >= 6`. El GOTO en el índice 8 retrocede al cuádruplo 2 para reevaluar la condición.

#	Op	Arg1	Arg2	Resultado
0	=	1	-	i
1	=	0	-	suma
2	<	i	6	t1
3	GOTOF	t1	-	9
4	+	suma	i	t2
5	=	t2	-	suma
6	+	i	1	t3
7	=	t3	-	i
8	GOTO	-	-	2
9	PRINT	suma	-	-

TEST 5. Función y llamada

Los cuádruplos 0-2 corresponden al cuerpo de la función `calcular`. Los cuádruplos 3-6 corresponden al cuerpo principal: asignaciones de `a` y `b`, dos cuádruplos `PARAM` para los argumentos, y el `CALL` a `calcular`.

#	Op	Arg1	Arg2	Resultado
0	+	x	y	t1
1	=	t1	-	temp
2	PRINT	temp	-	-
3	=	3	-	a
4	=	4	-	b
5	PARAM	a	-	-
6	PARAM	b	-	-
7	CALL	calcular	-	t2

10. Direcciones de memoria virtual

A continuación se documenta la distribución de las direcciones virtuales, para la cual se optó por bloques reservados para cada tipo de dato y scope.

Cuadro 15: Direcciones virtuales

Scope	Tipo	Inicio	Fin
Global	INT	1000	1099
Global	FLOAT	1100	1199
Global	BOOL	1200	1299
Local	INT	2000	2099
Local	FLOAT	2100	2199
Local	BOOL	2200	2299
Param	INT	3000	3099
Param	FLOAT	3100	3199
Param	BOOL	3200	3299
Temp	INT	4000	4099
Temp	FLOAT	4100	4199
Temp	BOOL	4200	4299
Const	INT	5000	5099
Const	FLOAT	5100	5199
Const	BOOL	5200	5299

11. Máquina Virtual

La Máquina Virtual de Patitoc interpreta la lista de cuádruplos generada por el compilador. Su arquitectura consta de tres estructuras principales: `MemoriaEjecucion`, `RegistroActivacion` y `MaquinaVirtual`.

11.1. Estructura: MemoriaEjecucion

Descripción

`MemoriaEjecucion` es una memoria de ejecución *plana* implementada sobre un diccionario Python `_mem: dict[int, object]`. Cada dirección virtual, asignada en tiempo de compilación, es la clave entera; su valor es el dato en tiempo de ejecución. La clase define cinco segmentos lógicos identificados exclusivamente por el rango numérico de la dirección.

Cuadro 16: Segmentos lógicos de `MemoriaEjecucion`

Segmento	Rango	Contenido
global	1000–1999	Variables globales del programa
local	2000–2999	Variables locales de funciones
param	3000–3999	Parámetros de funciones
temp	4000–4999	Temporales del compilador
const	5000–5999	Constantes literales (solo lectura)

Versión gráfica

Cuadro 17: Mapa de la MemoriaEjecucion y ejemplo de estado

Espacio de Direcciones			Snapshot en ejecución	
Rango	Segmento	Contenido típico	Addr	<code>_mem[addr]</code>
1000–1999	global	INT/FLOAT/BOOL globales	1000	10
2000–2999	local	INT/FLOAT/BOOL locales	2000	7
3000–3999	param	Parámetros de función	3000	3
4000–4999	temp	Temporales BOOL/INT/FLT	4200	True
5000–5999	const	Literales (read-only)	5000	10

El diccionario interno en un momento concreto de ejecución luce así:

```
_mem = {
    1000: 10,      # variable 'a', INT global
    1001: 5,       # variable 'b', INT global
    3000: 3,       # parámetro 'x', INT
    4200: True,    # temporal t1, BOOL
    5000: 10,      # constante literal 10
    5001: 5,       # constante literal 5
}
```

Métodos de acceso principales

Cuadro 18: Métodos de MemoriaEjecucion

Método	Costo	Descripción
<code>segmento(addr)</code>	$O(1)$	Recorre la lista de 5 segmentos y retorna el nombre del segmento al que pertenece <code>addr</code> . Lanza <code>RuntimeError</code> si la dirección está fuera de rango.
<code>leer(addr)</code>	$O(1)$	Valida el rango y retorna <code>_mem.get(addr, 0)</code> .
<code>escribir(addr, valor)</code>	$O(1)$	Verifica que <code>addr</code> no pertenezca al segmento <code>const</code> (protección de solo lectura) y actualiza <code>_mem[addr]</code> .
<code>precargar(addr, valor)</code>	$O(1)$	Carga constantes literales sin validar el segmento; invocado una sola vez al construir <code>MaquinaVirtual</code> .
<code>dump()</code>	$O(n)$	Retorna una cadena de depuración con todas las entradas del diccionario ordenadas por dirección, indicando su segmento.

11.2. Estructura: RegistroActivacion

Descripción

Cada llamada a función empuja un **RegistroActivacion** sobre la pila de llamadas de **MaquinaVirtual**. El registro almacena el estado mínimo necesario para restaurar la ejecución al retornar.

Versión gráfica

El siguiente esquema muestra el estado de la pila durante la ejecución de **fib(2)** (Fibonacci recursivo), donde tres frames están activos simultáneamente:

Cuadro 19: Estado de la pila de llamadas durante **fib(2)**

Frame	nombre	retorno_ip	retorno_dir	snapshot
tope (3er)	"fib"	12	4003	{2000:0, 2001:1}
2do	"fib"	12	4001	{2000:0, 2001:0}
base (1er)	"fib"	23	4005	{}

Campos

Cuadro 20: Campos de **RegistroActivacion**

Campo	Tipo	Descripción
nombre	str	Nombre de la función llamada. Usado para mensajes de error y depuración.
retorno_ip	int	Índice del cuádruplo al que retorna la ejecución después de ENDPROC o RETURN . Equivale a ip + 1 del cuádruplo CALL .
retorno_dir	int None	Dirección virtual donde se almacena el valor de retorno. Es None para funciones VOID .
snapshot	dict	Captura de los valores actuales de locales y parámetros tomada <i>antes</i> de sobrescribirlos con los de la nueva llamada. Se restaura con _mem.update(snapshot) al retornar, habilitando la recursión.

11.3. Estructura: MaquinaVirtual

Descripción

MaquinaVirtual es el ejecutor de cuádruplos. Mantiene un puntero de instrucción **ip** e itera sobre la lista de cuádruplos invocando **_despachar(op, arg1, arg2, res)** en cada paso.

Atributos principales

Cuadro 21: Atributos de MaquinaVirtual

Atributo	Tipo	Descripción
cuadрупlos	list[tuple]	Lista de cuádruplos (op, arg1, arg2, res) producidos por el compilador.
memoria	MemoriaEjecucion	Almacén plano de variables, parámetros, temporales y constantes.
pila_llamadas	list[Registro-Activacion]	Pila de frames activos; crece con CALL y decrece con RETURN/ENDPROC.
pila_params	list	Staging de argumentos: los valores evaluados por cuádruplos PARAM se acumulan aquí hasta el siguiente CALL.
ip	int	Puntero de instrucción; índice del cuádruplo en ejecución.

Ciclo de ejecución

```
def ejecutar(self) -> None:
    self.ip = 0
    while self.ip < len(self.cuadрупlos):
        op, arg1, arg2, res = self.cuadрупlos[self.ip]
        self._despachar(op, arg1, arg2, res)
```

Los opcodes soportados son: =, +, -, *, /, NEG, >, <, !=, ==, PRINT, GOTO, GOTOF, PARAM, CALL, RETURN, ENDPROC, END.

11.4. Cómo las Direcciones Virtuales Indexan la Memoria

El compilador asigna a cada símbolo una dirección virtual única en tiempo de compilación. Esta dirección es el único dato que la MV necesita para acceder a la memoria en tiempo de ejecución.

1. **Identificación de segmento sin metadato extra.** El rango numérico de la dirección identifica el segmento de forma fácil: 1000–1999 : `global`, 2000–2999 : `local`, 3000–3999 : `param`, 4000–4999 : `temp`, 5000–5999 : `const`.
2. **Acceso en $O(1)$ sin búsqueda por nombre.** Los cuádruplos contienen direcciones enteras en lugar de nombres (`a`, `t1`). La MV ejecuta `_mem[addr]` directamente, sin consultar el Directorio de Funciones ni la Tabla de Variables
3. **Constantes precargadas al iniciar.** Al construir `MaquinaVirtual`, todas las constantes literales registradas en `MemoriaVirtual` se copian al diccionario vía `precargar`, ocupando el segmento `const`

4. **Recursión habilitada por snapshot.** Las direcciones de locales y parámetros son únicas globalmente, por lo que llamadas recursivas sobrescribirían los valores de la invocación padre. Cada `RegistroActivacion` captura un `snapshot` antes de sobrescribir y lo restaura al retornar, logrando el efecto de un frame por invocación.

Cuadro 22: Ejemplo de traducción nombre $\rightarrow$ dirección virtual $\rightarrow$ clave en `_mem`

Símbolo	Categoría	Segmento	Dirección	En <code>_mem</code>
a (global INT)	VAR	global	1000	<code>_mem[1000] = 10</code>
b (global INT)	VAR	global	1001	<code>_mem[1001] = 5</code>
x (param INT)	PARAM	param	3000	<code>_mem[3000] = 3</code>
t1 (BOOL temp)	TEMP	temp	4200	<code>_mem[4200] = True</code>
"10" (const INT)	CONST	const	5000	<code>_mem[5000] = 10</code>

11.5. Casos de Prueba de la Máquina Virtual

A continuación se presentan los programas de prueba, sus cuádruplos reales con direcciones virtuales y la salida producida por la MV.

TEST 2 — Flotantes y PRINT con letrero

```
PROGRAM prog2 ;
VARS x : FLOAT ; n : INT ;
BEGIN {
    n = 4 ;
    x = 3.14 ;
    x = x * 2.0 + 1.0 ;
    PRINT( "Resultado: " , x ) ;
} END
```

Cuádruplos generados (con direcciones virtuales):

#	Op	Arg1	Arg2	Res	Nota
--	-----	-----	-----	-----	-----
0	GOTO	-	-	1	
1	=	5000	-	1000	n = const(4)
2	=	5100	-	1100	x = const(3.14)
3	*	1100	5101	4100	t1 = x * const(2.0)
4	+	4100	5102	4101	t2 = t1 + const(1.0)
5	=	4101	-	1100	x = t2
6	PRINT	5300	-	-	imprime "Resultado: "
7	PRINT	1100	-	-	imprime x
8	END	-	-	-	

Salida:

Resultado:
7.28

TEST 3 — Condicional IF-ELSE

```

PROGRAM prog3 ;
VARs a, b : INT ;
BEGIN {
    a = 10 ; b = 5 ;
    IF ( a > b ) { PRINT( "a es mayor" ) ; }
    ELSE          { PRINT( "b es mayor o igual" ) ; } ;
} END

```

#	Op	Arg1	Arg2	Res	Nota
--	-----	-----	-----	---	-----
0	GOTO	-	-	1	
1	=	5000	-	1000	a = 10
2	=	5001	-	1001	b = 5
3	>	1000	1001	4200	t1 = a > b
4	GOTOF	4200	-	7	si t1=False, salta al bloque ELSE
5	PRINT	5300	-	-	"a es mayor"
6	GOTO	-	-	8	salta el bloque ELSE
7	PRINT	5301	-	-	"b es mayor o igual"
8	END	-	-	-	

Salida:

a es mayor

TEST 4 — Ciclo WHILE

```

PROGRAM prog4 ;
VARs i, suma : INT ;
BEGIN {
    i = 1 ; suma = 0 ;
    WHILE ( i < 6 ) DO {
        suma = suma + i ;
        i = i + 1 ;
    } ;
    PRINT( suma ) ;
} END

```

Cuádruplos generados:

#	Op	Arg1	Arg2	Res	Nota
--	-----	-----	-----	---	-----
0	GOTO	-	-	1	
1	=	5000	-	1000	i = 1
2	=	5001	-	1001	suma = 0
3	<	1000	5002	4200	t1 = i < 6
4	GOTOF	4200	-	10	si t1=False, sale del ciclo
5	+	1001	1000	4000	t2 = suma + i
6	=	4000	-	1001	suma = t2

```

7  +      1000  5000  4001  t3  = i + 1
8  =      4001  -    1000  i    = t3
9  GOTO    -    -    3      regresa a evaluar la condición
10 PRINT   1001  -    -      imprime suma
11 END     -    -    -

```

Salida:

15

TEST 6 — Fibonacci Iterativo

```

PROGRAM fibonacci ;
VARs r : INT ;
INT fib ( num : INT ) {
  VARs a, b, temp, i : INT ;
  {
    a = 0 ; b = 1 ; i = 0 ;
    WHILE ( i < num ) DO {
      temp = a + b ; a = b ; b = temp ; i = i + 1 ;
    } ;
  }
  RETURN a ;
} ;
BEGIN {
  PRINT( "fib(0):" ) ; r = fib( 0 ) ; PRINT( r ) ;
  PRINT( "fib(5):" ) ; r = fib( 5 ) ; PRINT( r ) ;
  PRINT( "fib(10):" ) ; r = fib( 10 ) ; PRINT( r ) ;
} END

```

Salida:

```

fib(0):
0
fib(5):
5
fib(10):
55

```

TEST 7 — Fibonacci Recursivo

```

PROGRAM fibonacci_recursivo ;
VARs r : INT ;
INT fib ( n : INT ) {
  VARs a, b : INT ;
  {
    IF ( n == 0 ) { a = 0 ; } ELSE {
      IF ( n == 1 ) { a = 1 ; } ELSE {
        a = fib( n - 1 ) ;
        b = fib( n - 2 ) ;

```

```
        a = a + b ;
    } ;
}
RETURN a ;
} ;
BEGIN {
    PRINT( "fib(0):" ) ; r = fib( 0 ) ; PRINT( r ) ;
    PRINT( "fib(5):" ) ; r = fib( 5 ) ; PRINT( r ) ;
    PRINT( "fib(10):" ) ; r = fib( 10 ) ; PRINT( r ) ;
} END
```

Salida:

```
fib(0):
0
fib(5):
5
fib(10):
55
```

TEST 8 — Factorial Recursivo

```
PROGRAM factorial ;
VARs r : INT ;
INT fact ( n : INT ) {
    VARs res : INT ;
    {
        IF ( n == 0 ) { res = 1 ; } ELSE {
            res = n * fact( n - 1 ) ;
        } ;
    }
    RETURN res ;
} ;
BEGIN {
    PRINT( "fact(0):" ) ; r = fact( 0 ) ; PRINT( r ) ;
    PRINT( "fact(5):" ) ; r = fact( 5 ) ; PRINT( r ) ;
    PRINT( "fact(10):" ) ; r = fact( 10 ) ; PRINT( r ) ;
} END
```

Salida:

```
fact(0):
1
fact(5):
120
fact(10):
3628800
```

Declaración de Uso de Inteligencia Artificial

El presente documento fue elaborado con apoyo de la herramienta de inteligencia artificial Claude (Anthropic) como asistente en la redacción y estructuración del contenido en $\text{\LaTeX}$.

Las actividades en las que se utilizó la herramienta fueron:

- Generación y corrección de la plantilla base en $\text{\LaTeX}$ compatible con MiKTeX y LaTeX Workshop.
- Implementación de estructuras de datos básica del cubo semántico, Directorio de Funciones y Tablas de Variables.
- Generación del código en Python que implementa estructuras básicas de pilas y filas.
- Elaboración de las tablas de métodos de acceso y casos de prueba.

Todas las decisiones de diseño, definición de tokens, gramática y validaciones semánticas y documentación fueron revisadas y aprobadas por el autor, quien es responsable del contenido final del documento.

Carlos Fernando Ramos Mena
Matrícula: A01197622
8 de junio de 2026